

Experiment - 7.1.1. Inverted Star Pattern

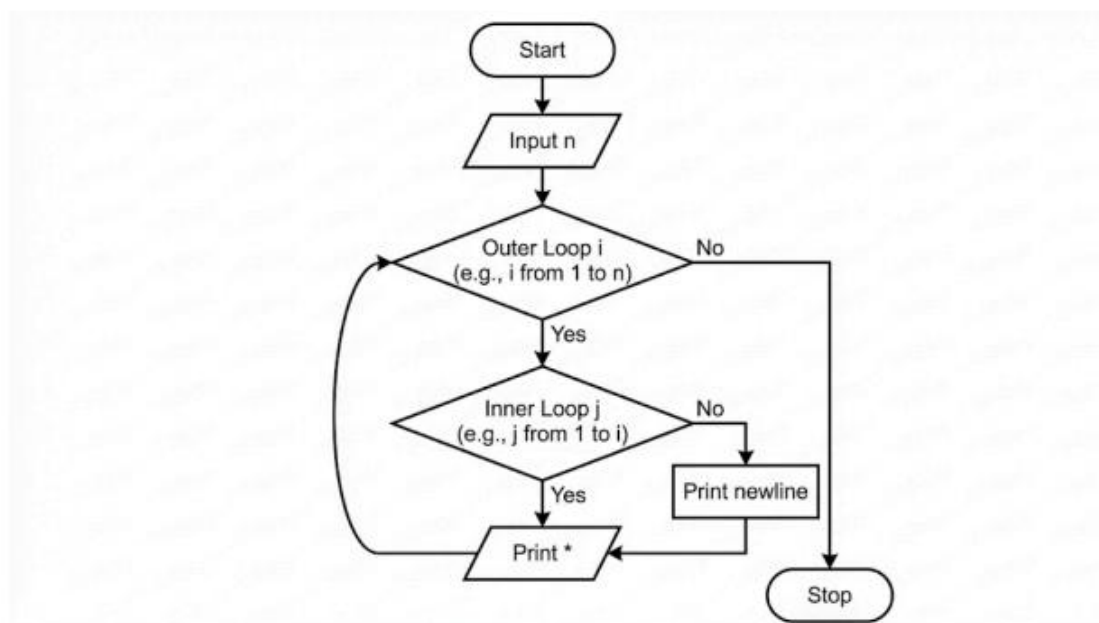
1. Aim

To design and implement a Python program that prints an inverted right-angled triangle star pattern. For a given integer n , the program uses nested loops to print n rows of stars, starting with n stars in the first row and decreasing by one star in each subsequent row until the last row has 1 star.

2. Pseudocode

1. **START**
2. **READ** a single integer from the user and **STORE** it in the variable n .
3. **FOR** each number i in the range starting from n down to 1 (decrementing by 1):
 - **FOR** each number j in the range from 0 to $i - 1$:
 - **PRINT** a star "*" without a newline character at the end.
 - **END INNER FOR**
 - **PRINT** an empty line (newline character) to move to the next row.
4. **END OUTER FOR**
5. **END**

3. Flowchart



7.1.1. Inverted Star Pattern

Write a Python program to print an inverted right-angled triangle star pattern.

For a given number n , the program should print n rows of stars where the first row contains n stars, the second row contains $n - 1$ stars, and each subsequent row has one less star than the previous row, ending with 1 star in the last row.

All rows should be left-aligned with no spaces between the stars.

Input Format:

- Single line contains an integer n representing the number of rows

Output Format:

- Print an inverted right-angled triangle star pattern with n rows

Note:

- Refer to sample test cases for better understanding.

Sample Test Cases

starPrint.py

```
1 n = int(input())
2 while (n > 0):
3     print("*" * n)
4     n = n - 1
```

Average time
0.003 s
3.17 ms

Maximum time
0.007 s
7.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 4 ms

Test case 2 4 ms

Expected output

3

**
*

Actual output

3

**
*

Terminal

Test cases